

SQL FULL COURSE

Basic → Intermediate → Advanced

Beginner-friendly explanations • Practical examples • Interview preparation • Advanced queries

Recommended database: MySQL / PostgreSQL / Oracle concepts are explained where syntax differs.

Practice database: Employee, Department, Project and Salary examples.

Goal: Learn SQL from zero and become comfortable writing real-world and interview queries.

Table of Contents

1. SQL and Database Fundamentals
2. Creating the Practice Database
3. SELECT and Filtering
4. INSERT, UPDATE and DELETE
5. Sorting, DISTINCT and Aliases
6. Aggregate Functions
7. GROUP BY and HAVING
8. Constraints and Keys
9. Joins
10. Subqueries
11. CASE Expressions
12. String Functions
13. Date and Time Functions
14. NULL Handling
15. Set Operators
16. Transactions and TCL
17. Views
18. Indexes
19. Common Table Expressions (CTEs)
20. Window Functions
21. Advanced SQL Patterns
22. Normalization
23. Stored Procedures, Functions, Cursors and Triggers
24. PL/SQL Basics (Oracle)
25. Interview Questions and Practice
26. SQL Cheat Sheet

1. SQL and Database Fundamentals

SQL (Structured Query Language) is used to store, retrieve, modify and analyze data in relational databases.

Database

A database is an organized collection of data. A relational database stores data in tables made of rows and columns.

Important terms

Table: collection of related data. **Row:** one record. **Column:** one attribute. **Primary key:** uniquely identifies a row.

Foreign key: connects one table to another.

Main SQL categories

DDL: CREATE, ALTER, DROP, TRUNCATE

DML: INSERT, UPDATE, DELETE

DQL: SELECT

DCL: GRANT, REVOKE

TCL: COMMIT, ROLLBACK, SAVEPOINT

SQL execution order

A useful logical order is: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET.

■ SQL keywords are generally written in uppercase for readability, but SQL is usually case-insensitive for keywords.

2. Creating the Practice Database

Create Department table

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY,  
  department_name VARCHAR(50) NOT NULL  
);
```

Create Employee table

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  employee_name VARCHAR(100) NOT NULL,  
  department_id INT,  
  salary DECIMAL(10,2),  
  hire_date DATE,  
  city VARCHAR(50),  
  manager_id INT,  
  FOREIGN KEY (department_id)  
  REFERENCES departments(department_id)  
);
```

Insert sample departments

```
INSERT INTO departments VALUES  
(10, 'IT'),  
(20, 'HR'),  
(30, 'Finance'),  
(40, 'Sales');
```

Insert sample employees

```
INSERT INTO employees  
(employee_id, employee_name, department_id, salary, hire_date, city, manager_id)
```

```
VALUES
(101, 'Ravi', 10, 65000, '2023-01-10', 'Hyderabad', NULL),
(102, 'Priya', 10, 80000, '2022-05-15', 'Bengaluru', 101),
(103, 'Arun', 20, 50000, '2024-02-20', 'Hyderabad', NULL),
(104, 'Sita', 30, 90000, 'Chennai', NULL),
(105, 'Kiran', 30, 70000, 'Hyderabad', 104),
(106, 'Meena', 40, 55000, 'Pune', NULL),
(107, 'Vijay', 10, 80000, 'Hyderabad', 101);
```

3. SELECT and Filtering

Select all columns

```
SELECT * FROM employees;
```

Select specific columns

```
SELECT employee_name, salary FROM employees;
```

WHERE

```
SELECT employee_name, salary
FROM employees
WHERE salary > 60000;
```

AND / OR

```
SELECT employee_name
FROM employees
WHERE department_id = 10
AND salary > 70000;
```

```
SELECT employee_name
FROM employees
WHERE city = 'Hyderabad'
OR city = 'Pune';
```

IN, BETWEEN, LIKE

```
SELECT * FROM employees
WHERE department_id IN (10, 30);
```

```
SELECT * FROM employees
WHERE salary BETWEEN 50000 AND 80000;
```

```
SELECT * FROM employees
WHERE employee_name LIKE 'P%';
```

LIKE patterns: 'A%' starts with A, '%a' ends with a, '%ar%' contains ar, '_' represents one character.

4. INSERT, UPDATE and DELETE

INSERT

```
INSERT INTO employees
(employee_id, employee_name, department_id, salary, hire_date, city)
VALUES (108, 'Anil', 20, 52000, '2025-01-05', 'Delhi');
```

UPDATE

```
UPDATE employees
SET salary = salary + 5000
WHERE employee_id = 108;
```

DELETE

```
DELETE FROM employees
WHERE employee_id = 108;
```

■ Always use a WHERE condition with UPDATE or DELETE unless you intentionally want to affect every row.

5. Sorting, DISTINCT and Aliases

```
SELECT employee_name, salary
FROM employees
ORDER BY salary DESC;
```

```
SELECT DISTINCT city
FROM employees;
```

```
SELECT employee_name AS name,
salary AS monthly_salary
FROM employees;
```

ASC means ascending and DESC means descending.

6. Aggregate Functions

Aggregate functions calculate one result from multiple rows.

```
SELECT COUNT(*) FROM employees;
SELECT COUNT(department_id) FROM employees;
SELECT SUM(salary) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MIN(salary) FROM employees;
SELECT MAX(salary) FROM employees;
```

Example

```
SELECT
COUNT(*) AS employee_count,
AVG(salary) AS average_salary,
MAX(salary) AS highest_salary
FROM employees;
```

7. GROUP BY and HAVING

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id;
```

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 65000;
```

WHERE filters rows before grouping. **HAVING** filters groups after aggregation.

8. Constraints and Keys

Common constraints are PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT.

```
CREATE TABLE students (
student_id INT PRIMARY KEY,
email VARCHAR(100) UNIQUE,
name VARCHAR(100) NOT NULL,
age INT CHECK (age >= 18),
city VARCHAR(50) DEFAULT 'Hyderabad'
);
```

Primary key: unique and not null. **Foreign key:** maintains a relationship. **Unique:** prevents duplicate values. **Check:** validates a condition.

9. Joins

INNER JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

Returns matching rows from both tables.

LEFT JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Returns all rows from the left table and matching rows from the right table.

RIGHT JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.department_id;
```

FULL OUTER JOIN

Returns matched and unmatched rows from both tables. MySQL does not support FULL OUTER JOIN directly; it can be simulated with UNION of LEFT and RIGHT joins.

CROSS JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

Produces every possible combination.

SELF JOIN

```
SELECT e.employee_name AS employee,
m.employee_name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.employee_id;
```

10. Subqueries

Employees earning above average

```
SELECT employee_name, salary
FROM employees
WHERE salary > (
SELECT AVG(salary)
FROM employees
);
```

IN subquery

```
SELECT employee_name
FROM employees
WHERE department_id IN (
SELECT department_id
FROM departments
WHERE department_name IN ('IT', 'Finance')
);
```

EXISTS

```
SELECT d.department_name
FROM departments d
WHERE EXISTS (
SELECT 1
FROM employees e
WHERE e.department_id = d.department_id
);
```

11. CASE Expressions

```
SELECT employee_name, salary,
CASE
WHEN salary >= 80000 THEN 'High'
WHEN salary >= 60000 THEN 'Medium'
ELSE 'Low'
END AS salary_level
FROM employees;
```

CASE is useful for conditional labels, categories and business rules.

12. String Functions

```
SELECT UPPER(employee_name) FROM employees;
SELECT LOWER(employee_name) FROM employees;
SELECT LENGTH(employee_name) FROM employees;
SELECT CONCAT(employee_name, ' - ', city) FROM employees;
SELECT TRIM(' SQL ');
SELECT SUBSTRING(employee_name, 1, 3) FROM employees;
```

Exact function names and parameters can differ slightly across MySQL, PostgreSQL and Oracle.

13. Date and Time Functions

```
SELECT CURRENT_DATE;
SELECT CURRENT_TIMESTAMP;

SELECT employee_name, hire_date
FROM employees
WHERE hire_date >= '2024-01-01';
```

Date difference examples

```
-- MySQL
SELECT employee_name,
DATEDIFF(CURRENT_DATE, hire_date) AS days_worked
FROM employees;
```

PostgreSQL and Oracle use different date functions. Always check the database dialect you are using.

14. NULL Handling

NULL means missing/unknown value. It is not equal to zero or an empty string.

```
SELECT *
FROM employees
WHERE manager_id IS NULL;

SELECT *
FROM employees
WHERE manager_id IS NOT NULL;
```

COALESCE

```
SELECT employee_name,
COALESCE(manager_id, 0) AS manager
FROM employees;
```

Use IS NULL / IS NOT NULL rather than = NULL.

15. Set Operators

```
SELECT city FROM employees
UNION
SELECT department_name FROM departments;
```

UNION removes duplicates. UNION ALL keeps duplicates.

```
SELECT city FROM employees
UNION ALL
SELECT city FROM another_table;
```

INTERSECT returns common rows where supported. Oracle commonly uses MINUS for set difference; PostgreSQL uses EXCEPT.

16. Transactions and TCL

A transaction is a logical unit of work.

```
UPDATE employees
SET salary = salary + 1000
WHERE department_id = 10;

COMMIT;
```

```
UPDATE employees
SET salary = salary + 1000
WHERE department_id = 10;

ROLLBACK;
```

```
SAVEPOINT before_change;
UPDATE employees
SET salary = salary + 2000
WHERE employee_id = 101;

ROLLBACK TO before_change;
```

COMMIT makes changes permanent. ROLLBACK undoes uncommitted changes. SAVEPOINT creates a point to roll back to.

17. Views

```
CREATE VIEW it_employees AS
SELECT employee_id, employee_name, salary
FROM employees
WHERE department_id = 10;

SELECT * FROM it_employees;
```

```
DROP VIEW it_employees;
```

A view is a stored query that behaves like a virtual table.

18. Indexes

```
CREATE INDEX idx_employee_salary  
ON employees(salary);
```

```
CREATE INDEX idx_employee_dept_salary  
ON employees(department_id, salary);
```

Indexes can speed up searches and joins, but they add storage and can make INSERT/UPDATE/DELETE more expensive. Index columns used frequently in filtering, joining or ordering after considering the actual workload.

■ Use EXPLAIN / EXPLAIN ANALYZE to inspect query plans, depending on your database.

19. Common Table Expressions (CTEs)

```
WITH high_salary AS (  
  SELECT employee_id, employee_name, salary  
  FROM employees  
  WHERE salary > 70000  
)  
SELECT *  
FROM high_salary;
```

Multiple CTEs

```
WITH dept_avg AS (  
  SELECT department_id, AVG(salary) AS avg_salary  
  FROM employees  
  GROUP BY department_id  
)  
SELECT e.employee_name, e.salary, d.avg_salary  
FROM employees e  
JOIN dept_avg d  
ON e.department_id = d.department_id  
WHERE e.salary > d.avg_salary;
```

CTEs improve readability and can break a complex query into logical steps.

20. Window Functions

ROW_NUMBER

```
SELECT employee_name, department_id, salary,  
ROW_NUMBER() OVER (  
  PARTITION BY department_id  
  ORDER BY salary DESC  
) AS rn  
FROM employees;
```

RANK and DENSE_RANK

```
SELECT employee_name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rnk,  
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk  
FROM employees;
```

RANK leaves gaps after ties; DENSE_RANK does not. ROW_NUMBER always gives unique sequential numbers within the window.

LEAD and LAG

```
SELECT employee_name, salary,
LAG(salary) OVER (ORDER BY salary) AS previous_salary,
LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

21. Advanced SQL Patterns

Second highest salary

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
SELECT MAX(salary)
FROM employees
);
```

Nth highest salary with DENSE_RANK

```
SELECT employee_name, salary
FROM (
SELECT employee_name, salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
) x
WHERE rnk = 3;
```

Top 2 salaries per department

```
SELECT employee_name, department_id, salary
FROM (
SELECT employee_name, department_id, salary,
DENSE_RANK() OVER (
PARTITION BY department_id
ORDER BY salary DESC
) AS rnk
FROM employees
) x
WHERE rnk <= 2;
```

Duplicate values

```
SELECT salary, COUNT(*) AS cnt
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

Employees with no manager

```
SELECT employee_name
FROM employees
WHERE manager_id IS NULL;
```

Employees earning more than their manager

```
SELECT e.employee_name, e.salary,
m.employee_name AS manager_name,
m.salary AS manager_salary
FROM employees e
JOIN employees m
ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

22. Normalization

Normalization organizes data to reduce duplication and update anomalies.

1NF

Values should be atomic; avoid storing multiple values in one column.

2NF

Be in 1NF and remove partial dependency on part of a composite key.

3NF

Be in 2NF and remove transitive dependencies.

Denormalization

Intentionally adding redundancy can improve read performance in some systems, especially analytical workloads, but it should be a deliberate design decision.

23. Stored Procedures, Functions, Cursors and Triggers

Procedure — Oracle example

```
CREATE OR REPLACE PROCEDURE give_raise (  
  p_employee_id IN NUMBER,  
  p_amount IN NUMBER  
)  
AS  
BEGIN  
  UPDATE employees  
  SET salary = salary + p_amount  
  WHERE employee_id = p_employee_id;  
END;  
/
```

Function — Oracle example

```
CREATE OR REPLACE FUNCTION get_salary (  
  p_employee_id IN NUMBER  
) RETURN NUMBER  
AS  
  v_salary NUMBER;  
BEGIN  
  SELECT salary  
  INTO v_salary  
  FROM employees  
  WHERE employee_id = p_employee_id;  
  
  RETURN v_salary;  
END;  
/
```

Cursor — concept

A cursor processes query results row by row. Prefer set-based SQL when possible; cursors are useful when procedural row-by-row logic is genuinely required.

Trigger — Oracle example

```
CREATE OR REPLACE TRIGGER trg_salary_audit  
AFTER UPDATE OF salary ON employees  
FOR EACH ROW  
BEGIN  
  INSERT INTO salary_audit(employee_id, old_salary, new_salary)  
  VALUES (:OLD.employee_id, :OLD.salary, :NEW.salary);  
END;
```

```
/
```

■ Procedure/function/trigger syntax varies significantly by database. The examples in this section are Oracle PL/SQL.

24. PL/SQL Basics (Oracle)

Variables and IF

```
DECLARE
v_salary NUMBER := 70000;
BEGIN
IF v_salary >= 60000 THEN
DBMS_OUTPUT.PUT_LINE('High salary');
ELSE
DBMS_OUTPUT.PUT_LINE('Lower salary');
END IF;
END;
/
```

LOOP

```
DECLARE
i NUMBER := 1;
BEGIN
LOOP
DBMS_OUTPUT.PUT_LINE(i);
i := i + 1;
EXIT WHEN i > 5;
END LOOP;
END;
/
```

SELECT INTO

```
DECLARE
v_name employees.employee_name%TYPE;
BEGIN
SELECT employee_name
INTO v_name
FROM employees
WHERE employee_id = 101;

DBMS_OUTPUT.PUT_LINE(v_name);
END;
/
```

Exception handling

```
BEGIN
-- statements
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No employee found');
WHEN TOO_MANY_ROWS THEN
DBMS_OUTPUT.PUT_LINE('More than one row');
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Other error');
END;
/
```

25. SQL Interview Questions and Practice

1. What is SQL?

A language used to work with relational databases: querying, inserting, updating, deleting and defining data.

2. DELETE vs TRUNCATE vs DROP?

DELETE removes selected rows and can use WHERE. TRUNCATE removes all rows quickly and generally cannot use WHERE. DROP removes the table object itself. Transaction behavior varies by database.

3. WHERE vs HAVING?

WHERE filters individual rows before grouping; HAVING filters grouped results after aggregation.

4. PRIMARY KEY vs UNIQUE?

A primary key identifies each row and cannot be NULL. UNIQUE prevents duplicate values; NULL behavior can vary by database.

5. INNER JOIN vs LEFT JOIN?

INNER JOIN returns matching rows. LEFT JOIN keeps every row from the left table and adds matches from the right.

6. RANK vs DENSE_RANK vs ROW_NUMBER?

RANK gives gaps after ties; DENSE_RANK does not; ROW_NUMBER assigns a unique sequence.

7. What is a subquery?

A query nested inside another query.

8. What is a CTE?

A named temporary result defined with WITH for one SQL statement.

9. What is an index?

A data structure that can speed up data access for suitable queries, with storage and write-maintenance costs.

10. What is normalization?

Organizing relational data to reduce redundancy and anomalies.

Practice set — Beginner

1. Display all employees.
2. Display employee names and salaries.
3. Find employees with salary greater than 60000.
4. Find employees from Hyderabad.
5. Find employees whose name starts with A.
6. Display unique cities.
7. Sort employees by salary descending.
8. Find the highest salary.
9. Find the average salary.
10. Count employees in each department.

Practice set — Intermediate

11. Display employee name with department name.

12. Find departments with more than 2 employees.
13. Find employees earning above the company average.
14. Find employees with no manager.
15. Find the second highest salary.
16. Find duplicate salaries.
17. Find the highest-paid employee in each department.
18. Find employees earning more than their manager.
19. Find departments that have at least one employee.
20. Find employees hired after 2023.

Practice set — Advanced

21. Find the top 3 salaries in every department.
22. Find the third distinct highest salary.
23. Rank employees within each department.
24. Compare each employee salary with the previous employee's salary.
25. Find employees whose salary is above their department average.
26. Find departments whose average salary is above the company average.
27. Find the percentage contribution of each department to total salary.
28. Use a CTE to identify high-salary employees.
29. Use a recursive CTE to display an employee-manager hierarchy (where supported).
30. Analyze a slow query with EXPLAIN/EXPLAIN ANALYZE.

26. SQL Cheat Sheet

Basic commands

```
SELECT * FROM table_name;

INSERT INTO table_name (col1, col2)
VALUES (value1, value2);

UPDATE table_name
SET col1 = value1
WHERE condition;

DELETE FROM table_name
WHERE condition;
```

Filtering

```
WHERE
AND
OR
IN
BETWEEN
LIKE
IS NULL
IS NOT NULL
```

Aggregation

```
COUNT()  
SUM()  
AVG()  
MIN()  
MAX()  
GROUP BY  
HAVING
```

Joins

```
INNER JOIN  
LEFT JOIN  
RIGHT JOIN  
FULL OUTER JOIN  
CROSS JOIN  
SELF JOIN
```

Advanced

```
WITH ... AS (...) -- CTE  
ROW_NUMBER() OVER (...)  
RANK() OVER (...)  
DENSE_RANK() OVER (...)  
LAG() OVER (...)  
LEAD() OVER (...)  
CASE WHEN ... END  
EXISTS  
EXPLAIN / EXPLAIN ANALYZE
```

Study order: SELECT → WHERE → ORDER BY → GROUP BY → HAVING → JOINS → SUBQUERIES → CASE → FUNCTIONS → SET OPERATORS → TRANSACTIONS → VIEWS → INDEXES → CTEs → WINDOW FUNCTIONS → ADVANCED QUERIES → PL/SQL.

Interview tip: Do not only memorize queries. Understand what each clause does, why it is used, and how the result changes when you modify it.

Final Revision

For a fresher SQL interview, focus strongly on SELECT, WHERE, ORDER BY, GROUP BY, HAVING, aggregate functions, joins, subqueries, CASE, NULL handling, CTEs and window functions.

Practice writing queries without copying them. Start with the employee tables in this PDF and gradually solve the advanced exercises.

Key rule: SQL becomes easy through repeated hands-on practice. Write the query, predict the result, execute it, and then explain why the result is correct.